

DESARROLLO WEB BACKEND • INF560

PRÁCTICA 6

API REST con Laravel, PostgreSQL y autenticación por tokens (Sanctum)

Trabajo Práctico Evaluado

DATOS DE LA TAREA

Materia: Desarrollo Web Backend (INF560) — 5.º semestre**Carrera:** Ingeniería Informática — Facultad de Ciencias Puras, UATF**Docente:** M.Sc. Huáscar Fedor Gonzales Guzmán**Modalidad:** Individual**Ponderación:** Sobre 100 puntos

COMPETENCIA A DESARROLLAR

Diseña, implementa y prueba una API REST en Laravel con persistencia en PostgreSQL, aplicando autenticación por tokens con Laravel Sanctum, validación de datos, autorización por usuario y buenas prácticas RESTful, para construir el backend de una aplicación de gestión de finanzas personales consumible por cualquier cliente HTTP.

1. Contexto del proyecto: FinTrack API

Una startup local desea ofrecer una aplicación móvil de finanzas personales llamada FinTrack. El equipo frontend desarrollará la app, pero necesita un backend que exponga todos los datos a través de una API REST segura. Tu tarea consiste en construir ese backend.

Cada usuario debe poder registrarse, iniciar sesión y administrar sus propias categorías y transacciones (ingresos y egresos). La aplicación también debe entregar un resumen financiero con totales y balance. Un punto crítico es que ningún usuario debe poder ver ni modificar los datos de otro usuario.

EJEMPLO DE USO REAL

- 1) Ana se registra → recibe un token de acceso.
- 2) Crea categorías: «Sueldo» (ingreso) y «Comida» (egreso).
- 3) Registra transacciones asociadas a esas categorías.
- 4) Consulta GET /api/summary y obtiene: ingresos, egresos y balance del mes.

2. Objetivos

Objetivo general

Construir una API REST funcional, segura y bien estructurada en Laravel con PostgreSQL, aplicando autenticación basada en tokens de Sanctum y los principios de diseño RESTful.

Objetivos específicos

- Configurar un proyecto Laravel conectado a una base de datos PostgreSQL.
- Modelar las entidades del dominio con migraciones y relaciones Eloquent.
- Implementar registro, inicio y cierre de sesión emitiendo y revocando tokens con Sanctum.
- Proteger rutas con el middleware auth:sanctum y aislar los datos por usuario.
- Implementar operaciones CRUD RESTful con códigos de estado HTTP correctos.
- Validar la entrada mediante Form Requests y dar formato a la salida con API Resources.
- Documentar y probar la API con una colección de Postman.

3. Stack tecnológico obligatorio

Componente	Requisito
Framework	Laravel 13
Lenguaje	PHP 8.3 o superior
Base de datos	PostgreSQL (driver pgsql)
Autenticación	Laravel Sanctum (API tokens)
Pruebas / cliente	Postman (colección exportada en el repositorio)
Control de versiones	Git + GitHub (repositorio público o con acceso al docente)

4. Modelo de datos

El dominio tiene tres entidades. `users` es la tabla por defecto de Laravel; debes crear `categories` y `transactions`.

categories

Campo	Tipo	Notas
<code>id</code>	<code>bigint</code>	PK autoincremental
<code>user_id</code>	<code>bigint</code>	FK → <code>users</code> (onDelete cascade)
<code>name</code>	<code>string</code>	Nombre de la categoría
<code>type</code>	<code>enum/string</code>	ingreso egreso
<code>color</code>	<code>string</code> (nullable)	Color HEX opcional para la UI
<code>timestamps</code>	—	<code>created_at</code> / <code>updated_at</code>

transactions

Campo	Tipo	Notas
<code>id</code>	<code>bigint</code>	PK autoincremental
<code>user_id</code>	<code>bigint</code>	FK → <code>users</code> (onDelete cascade)
<code>category_id</code>	<code>bigint</code>	FK → <code>categories</code>
<code>amount</code>	<code>decimal(12,2)</code>	Monto (> 0)
<code>description</code>	<code>string</code> (nullable)	Detalle de la transacción
<code>transaction_date</code>	<code>date</code>	Fecha de la transacción
<code>timestamps</code>	—	<code>created_at</code> / <code>updated_at</code>

RELACIONES ELOQUENT ESPERADAS

User: `hasMany(Category)`, `hasMany(Transaction)`.

Category: `belongsTo(User)`, `hasMany(Transaction)`.

Transaction: `belongsTo(User)`, `belongsTo(Category)`.

5. Especificación de endpoints

Todas las rutas viven bajo el prefijo `/api`. La columna «Auth» indica si la ruta exige token de Sanctum.

5.1 Autenticación

Método	Endpoint	Auth	Descripción
POST	<code>/api/register</code>	No	Registra un usuario y devuelve un token (201).
POST	<code>/api/login</code>	No	Valida credenciales y devuelve un token (200).
POST	<code>/api/logout</code>	Sí	Revoca el token actual (200).
GET	<code>/api/user</code>	Sí	Devuelve el usuario autenticado.

5.2 Categorías

Método	Endpoint	Auth	Descripción
GET	/api/categories	Sí	Lista las categorías del usuario.
POST	/api/categories	Sí	Crea una categoría (201).
GET	/api/categories/{id}	Sí	Muestra una categoría propia.
PUT	/api/categories/{id}	Sí	Actualiza una categoría propia.
DELETE	/api/categories/{id}	Sí	Elimina una categoría propia (204).

5.3 Transacciones

Método	Endpoint	Auth	Descripción
GET	/api/transactions	Sí	Lista transacciones. Filtros: ?type=&category_id=&from=&to=
POST	/api/transactions	Sí	Crea una transacción (201).
GET	/api/transactions/{id}	Sí	Muestra una transacción propia.
PUT	/api/transactions/{id}	Sí	Actualiza una transacción propia.
DELETE	/api/transactions/{id}	Sí	Elimina una transacción propia (204).

5.4 Resumen

Método	Endpoint	Auth	Descripción
GET	/api/summary	Sí	Totales de ingresos, egresos y balance. Filtros: ?from=&to=

8. Criterios de evaluación (rúbrica)

Criterio	Puntos
Configuración del proyecto y conexión a PostgreSQL	10
Migraciones y modelos con relaciones Eloquent	10
Autenticación con Sanctum (register / login / logout)	20
CRUD RESTful de categorías y transacciones	20
Aislamiento de datos por usuario (autorización)	10
Validación con Form Requests y códigos HTTP correctos	10
API Resources (formato de salida consistente)	5
Endpoint de resumen financiero	5
Colección de Postman y documentación (README)	10
TOTAL	100

10. Puntos extra (opcional, hasta +15)

- **Paginación y filtros:** GET /api/transactions con paginación y filtros combinables por tipo, categoría y rango de fechas.
- **Token abilities:** asignar habilidades/permisos a los tokens y verificarlos en los endpoints.
- **Rate limiting:** limitar peticiones por usuario con throttle.
- **Pruebas automatizadas:** tests de feature con PHPUnit o Pest sobre los flujos de autenticación y CRUD.

PISTA FINAL

Empieza por la autenticación y pruébala en Postman antes de construir el CRUD: si el token funciona, el resto fluye.

Verifica el aislamiento creando DOS usuarios e intentando, desde uno, acceder a los recursos del otro: debe fallar.